Site Reliability Engineering Basics: The Essential Beginner Cheatsheet

Site Reliability Engineering (SRE) bridges software development and operations to keep systems fast and reliable. Learning these foundational tasks helps you debug production issues faster and build resilient applications.

What you'll learn:

1. Checking Service Health with Curl
2. Triage Production Logs in Real Time
3. Debugging Network Ports and Connectivity
4. Undoing Bad Deployments with Kubernetes

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Checking Service Health with Curl

Site Reliability Engineers constantly monitor HTTP status codes to verify that services are healthy. Using curl with header flags allows you to inspect server responses quickly without downloading the full page body. This is the fastest way to confirm whether a downtime issue is client-side or server-side.

```
$curl -I -s -o /dev/null -w "HTTP Status: %{http_code}\nTotal Time: %{time_total}s\n" https://httpbin
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Triage Production Logs in Real Time

When an application fails, logs provide the immediate digital trail. By combining the tail command with grep, SREs filter out normal traffic noise and stream only critical error messages to their terminal. This helps identify root causes during active outages without searching massive files manually.

```
$tail -f /var/log/syslog | grep -E -i "error|critical|fatal"
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Debugging Network Ports and Connectivity

A common cause of downtime is blocked network access between microservices and databases. Netcat (nc) lets you test TCP port connectivity from one server to another instantly. If the connection fails, you know the issue lies in network security groups or firewalls.

```
$nc -zvw3 db.internal.net 5432
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Undoing Bad Deployments with Kubernetes

Even well-tested code can fail in production. SREs prioritize speed of recovery over immediate bug fixing during an incident. Using Kubernetes rollout undo instantly switches traffic back to the previous stable release while developers fix the bug safely in staging.

```
$kubectl rollout undo deployment/web-app --to-revision=1
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always check application logs before restarting a crashing server or container.
+ Set up automated alerts for high CPU, memory exhaustion, and 5xx HTTP error spikes.
+ Document incident steps in simple post-mortems so team members learn from outages.
+ Test emergency rollback procedures in staging before you need them in production.
+ Automate repetitive manual operational tasks using simple Bash or Python scripts.

Watch Out For

! Ignoring low-severity warning alerts until they pile up and turn into major outages.
! Making unrecorded manual changes directly on live production servers.
! Forgetting to set resource limits on containers, leading to out-of-memory node crashes.
! Treating temporary symptoms during an outage without finding the underlying root cause.

Follow for daily Cloud & DevOps guides!
linkedin.com/in/mramanrajsingh